tells you the type of the exception, followed by the actual exception message. This is repeated in bold at the top of the Stack tab content, followed by the name of the method that was called at the point the exception was raised:

```
CityBreaks.Pages.Pages_City.ExecuteAsync() in City.cshtml
```

The `ExecuteAsync` method belongs to the compiled Razor page. It is called at run time. The following listing shows how the start of it looks for your compiled City page.

Listing 12.2 The `ExecuteAsync` method for the compiled City page

```
public override async Task ExecuteAsync()
{
    this.WriteLiteral("\r\n");
    this.ViewData["Title"] =
    ➥ string.Concat(this.Model.City.Name, " Details");     ❶
    this.WriteLiteral("<h3 b-jbmzjjkv6t>");
    this.Write(this.Model.City.Name);
    this.WriteLiteral("</h3>\r\n");
```

❶ This line of code caused the exception to be raised.

NOTE In previous versions of Razor Pages, it is possible to easily see the C# code that is generated from a Razor page by navigating to /obj /Debug /[.Net app version] /Razor/Pages, where you will find a collection of files with a .g.cs extension. Each of these contains the generated source code for the matching Razor page. In .NET 6, the Razor compilation system was changed to support hot reload, and these intermediate files are no longer generated. Tools are available, such as the free JustDecompile from Telerik (https://www.telerik.com/products/decompiler.aspx), that enable you to decompile the generated assembly (.dll file) created by the build process. You can find the assembly in \bin\Debug \[.NET app version] \.

The `string.Concat` method in the `ExecuteAsync` method was generated by the Razor parser from your original interpolated string:

```
ViewData["Title"] = $"{Model.City.Name} Details";
```